

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer:

The Driver is the main program that starts the Spark application. It creates the SparkSession, prepares the execution plan, and sends tasks to the executors. The Cluster Manager is responsible for managing the available resources in the cluster and assigning executors. The Executors are worker processes that execute the assigned tasks, process data, and send the results back to the Driver.

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer:

Spark does not execute transformations immediately. Instead, it records all the operations and creates a Directed Acyclic Graph (DAG). The execution starts only when an action such as show() or count() is called. This allows Spark to optimize the execution plan, reduce unnecessary operations, and improve overall performance.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read.csv("data/source.csv", header=True, inferSchema=True)
```

Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer:

CSV is a row-based text file format, while Parquet is a columnar binary format. CSV files are usually larger and slower to process because Spark has to read the entire file. Parquet stores data by columns, supports compression, and keeps schema information. This allows Spark to read only the required columns, making data processing much faster and more efficient.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```
df.filter(df.category == "Electronics") \
```

```
.select("product_id", "price") \
.show()
```

Q6: Write the code to revise a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

```
from pyspark.sql.functions import col
```

```
df = df.withColumnRenamed("old_name", "new_name")
```

```
df = df.withColumn("price", col("price").cast("double"))
```

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Answer:

Spark keeps track of every transformation using a Lineage Graph. If a worker node fails and some data is lost, Spark uses this graph to recreate only the missing partitions from the original data instead of processing everything again. This helps Spark recover automatically without losing the entire job.

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

```
df_orders.filter(
    (df_orders.status == "Completed") &
    (df_orders.amount > 1000)
).show()
```

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Answer:

Predicate Pushdown allows Spark to apply filter conditions while reading a Parquet file. Instead of loading the entire dataset, Spark reads only the rows that satisfy the condition. This reduces memory usage, decreases disk I/O, and improves query performance.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

```
from pyspark.sql.functions import col
```

```
df = df.withColumn("final_price", col("base_price") * 1.18)
```

```
df.show()
```

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Answer:

Transformations are operations that create a new DataFrame but are not executed immediately. Spark stores these operations until an action is called.

Examples of transformations are:

- `filter()`
- `select()`

Actions trigger the execution of all pending transformations and produce a result.

Examples of actions are:

- `show()`
- `count()`

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where `user_id` is null, and save the result as a CSV at "path/to/output".

```
spark.read.parquet("path/to/input") \
```

```
.filter("user_id IS NOT NULL") \  
.write.mode("overwrite") \  
.option("header", True) \  
.csv("path/to/output")
```

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Answer:

In Client Mode, the Driver runs on the user's local machine while the executors run on the cluster. This mode is mainly used for development and testing.

In Cluster Mode, both the Driver and Executors run inside the cluster. It is more suitable for production because the application continues running even if the user disconnects.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
df.filter(  
    (df.region == "North") |  
    (df.priority == "High")  
)
```

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Answer:

The `show(5)` method displays only the first five rows of the dataset, so only a small amount of data is sent to the Driver. On the other hand, `collect()` transfers the entire dataset to the Driver's memory. For very large datasets, this can consume a huge amount of memory and may even crash the application. Therefore, `show(5)` is the safer and more efficient option for exploring large datasets.